IRELE

Advanced Measurement and Data Driven Modelling

Lab report

Authors:

Arico Amaury

Colot Emmeran

Gueye Sidy

Professor:

J. Lataire

TA:

F. Vandeputte

Academic year:

2024 - 2025

Contents

1

Lab 1: RLS with forgetting factor

1.1 Main steps

1. Measure the input and output signals from the DUT. The input is arbitrary and the time varying part of the system is due to a manual switch.
2. Make a first estimate of the parameters with an LLS estimator
3. Use the RLS estimator with forgetting factor to track the time-varying parameters of the system.
4. Deduce from the estimated parameters the time instants at which the system changes.

1.2 Time-varying RLS estimation

Starting from the recursive equation of the system, a matrix formulation is derived:

$$
\begin{aligned}
y(N) &= b_0(N)u(N) + b_1(N)u(N-1) - a_1(N)y(N-1) - a_2(N)y(N-2) && (1.1)\\
&= K(N)\theta(N) && (1.2)
\end{aligned}
$$

with

$$
K(N) = \begin{bmatrix} u(N) & u(N-1) & -y(N-1) & -y(N-2) \end{bmatrix}, \qquad \theta(N) = \begin{bmatrix} b_0(N) \\ b_1(N) \\ a_1(N) \\ a_2(N) \end{bmatrix} \tag{1.3}
$$

The RLS estimator (with forgetting factor) is a recursive algorithm that computes the parameters $\theta(N)$ and their covariance matrix K_N after each new sample is collected. Because it is recursive, it needs to be initialized and this is done with a linear least squares estimate on a few first samples.
The linear least squares estimator, used to find θ in $y_{\text{lls}} = H\theta$, is in our case constructed as follows:

$$H = \begin{bmatrix} u(n_{\text{start}}+1) & u(n_{\text{start}}) & -y(n_{\text{start}}) & -y(n_{\text{start}}-1) \\ u(n_{\text{start}}+2) & u(n_{\text{start}}+1) & -y(n_{\text{start}}+1) & -y(n_{\text{start}}) \\ \vdots & \vdots & \vdots & \vdots \\ u(n_{\text{LLS}}-1) & u(n_{\text{LLS}}-2) & -y(n_{\text{LLS}}-2) & -y(n_{\text{LLS}}-3) \end{bmatrix}, \qquad y_{\text{lls}} = \begin{bmatrix} y(n_{\text{start}}+1) \\ y(n_{\text{start}}+2) \\ \vdots \\ y(n_{\text{LLS}}-1) \end{bmatrix} \tag{1.4}$$

Where $n_{\text{start}} = \max(n_a, n_b)$ and n_{LLS} is the number of samples used for the LLS estimate, chosen to be 200 here.

$\theta(n_{\text{LLS}})$ and $P_{n_{\text{LLS}}}$ are found with, respectively, the solution to the LLS problem and the definition of the covariance matrix in the RLS algorithm:

$$\theta(n_{\text{LLS}}) = (H^T H)^{-1} H^T y_{\text{lls}}, \qquad P_{n_{\text{LLS}}} = (H^T H)^{-1} \tag{1.5}$$

The Matlab code of the LLS initialization is given below:

```
% initialization with an LLS estimate over a few of the first samples
% y_lls = H * theta
H = zeros(sampleForLLS - max(na, nb), na + nb + 1);
for uCol = 0:nb
    H(:, uCol + 1) = u((max(na, nb) + 1 - uCol:sampleForLLS - uCol));
end
for yCol = 1:na
    H(:, nb + 1 + yCol) = -y((max(na, nb) + 1 - yCol:sampleForLLS - yCol));
end
y_lls = y((max(na, nb) + 1:sampleForLLS));

theta = H\y_lls;
P = inv(H'*H);
```

The recursive least squares equations with forgetting factor g are given by:

$$\theta(N) = \theta(N-1) - \frac{P_{N-1}K^T(N)\left[K(N)\theta(N-1) - y(N)\right]}{g + K(N)P_{N-1}K^T(N)} \tag{1.6}$$

$$P_N = \frac{1}{g}\left(P_{N-1} - \frac{P_{N-1}K^T(N)K(N)P_{N-1}}{g + K(N)P_{N-1}K^T(N)}\right) \tag{1.7}$$

The Matlab code implementing these equations is given below:

```
theta_sav = zeros(na + nb + 1, N - max(na, nb) + 2);
P_diag_sav = zeros(na + nb + 1, N - max(na, nb) + 2);

for itr = sampleForLLS+1:N
```

```
    % saving previous values
    theta_sav(:, itr - max(na, nb) + 1) = theta;
    P_diag_sav(:, itr - max(na, nb) + 1) = diag(P);

    K_itr = [u(itr - (0:nb)), -y(itr - (1:na))];

    theta = theta - (P * (K_itr') * (K_itr * theta - y(itr))) / (g + K_itr * P * (K_itr'));
    P = 1/g * (P - (P * (K_itr') * K_itr * P) / (g + K_itr * P * (K_itr')));

end
```

1.3 Results

The input of the system is not interesting to show here as it is random white noise. The evolution in time of the estimated parameters is however much more interesting:

Plot theta(g) for at least 2 experiments

Conclude which g suits the best

From the results above, we can take as estimated parameters the values just before the switch and at then end of the experiment. Because the system consists of two LTI systems switched manually, the transfer functions of each of them can be deduced from the estimated parameters:

$$G_{low}(z) = \frac{b_0 z + b_1}{z^2 + a_1 z + a_2} = \frac{0.5z + 0.15}{z^2 - 0.4z + 0.05} \tag{1.8}$$

$$G_{high}(z) = \frac{b_0 z + b_1}{z^2 + a_1 z + a_2} = \frac{0.29z + 0.27}{z^2 - 1.25z + 0.8} \tag{1.9}$$

and their frequency responses can be plotted as follows:

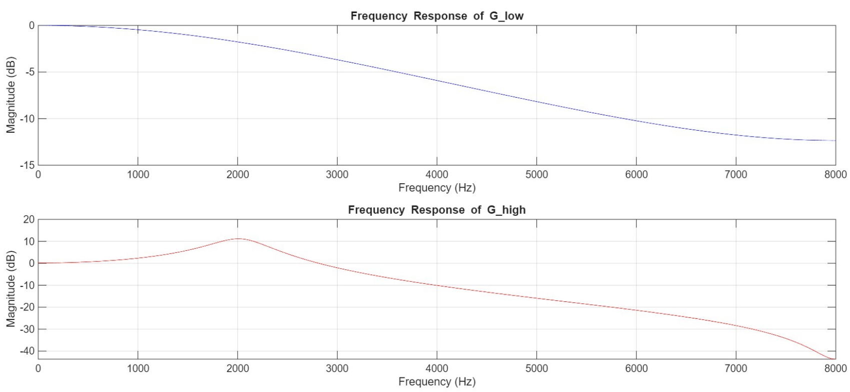

Figure 1.1: Frequency responses of the two operating modes of the system

1.4 Conclusion

See then

2

Lab 2: Kalman filter

2.1 Main steps

1. TODO

2.2 Continuous-time state space model

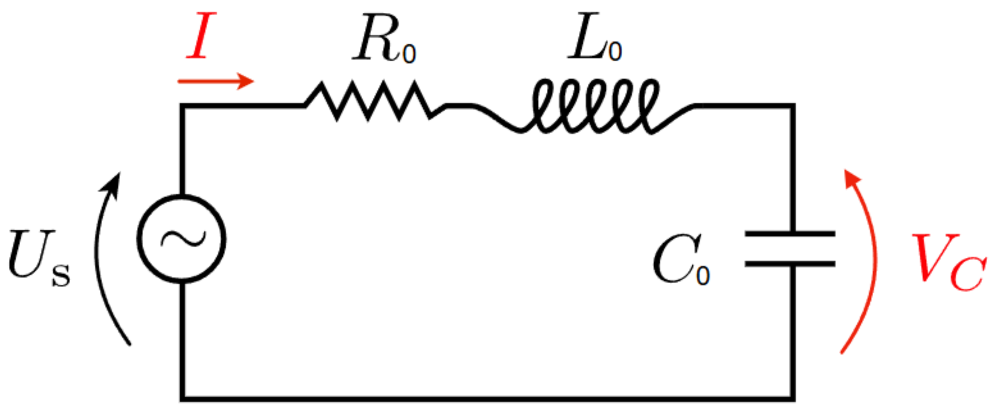

Figure 2.1: Studied RLC circuit

The studied system is the RLC circuit shown in Fig 2.1. A set of equations can be derived using Kirchhoff's laws:

$$u_s(t) = R_0\, i(t) + L_0 \frac{di(t)}{dt} + \frac{1}{C_0}\int i(t)\, dt \tag{2.1}$$

$$v_C(t) = \frac{1}{C_0}\int i(t)\, dt \tag{2.2}$$

By substituting the second equation into the first one, we obtain:

$$u_s(t) = R_0\, i(t) + L_0 \frac{di(t)}{dt} + v_C(t) \tag{2.3}$$

To get to the continuous-time state space representation of this circuit, we need to define the input of the system as $u_s(t)$, its output as $v_C(t)$ and the state vector as:

$$x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} = \begin{bmatrix} v_C(t) \\ i(t) \end{bmatrix} \tag{2.4}$$

Which finally leads to:

$$\frac{d}{dt}\begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} = \begin{bmatrix} 0 & \frac{1}{C_0} \\ -\frac{1}{L_0} & -\frac{R_0}{L_0} \end{bmatrix} \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{1}{L_0} \end{bmatrix} u_s(t) \tag{2.5}$$

$$y(t) = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} \tag{2.6}$$

$$\Leftrightarrow \qquad A_c = \begin{bmatrix} 0 & \frac{1}{C_0} \\ -\frac{1}{L_0} & -\frac{R_0}{L_0} \end{bmatrix}, \qquad B_c = \begin{bmatrix} 0 \\ \frac{1}{L_0} \end{bmatrix}, \qquad C_c = \begin{bmatrix} 1 & 0 \end{bmatrix}, \qquad D_c = 0 \tag{2.7}$$

2.3 Discrete-time state space model

To convert this continuous-time state space model to a discrete-time one, we need to select a sampling frequency. As the poles of the system are the eigenvalues of the matrix A_c, we can compute them with Matlab, giving:

$$\begin{cases} \lambda_1 \approx -1\text{MHz} + j8.1 \\ \lambda_2 \approx -1\text{MHz} - j8.1 \end{cases} \tag{2.8}$$

The band of interest is therefore chosen to be higher than 1 MHz in order to capture the dynamics of the system. the sampling frequency is thus set to f_s = 4 MHz. The discrete-time state space matrices are computed with Matlab. Two different methods are used: ZOH and tustin.
To compare which method is closer to the continuous-time model, the step response of the three models are plotted in Fig 2.2.

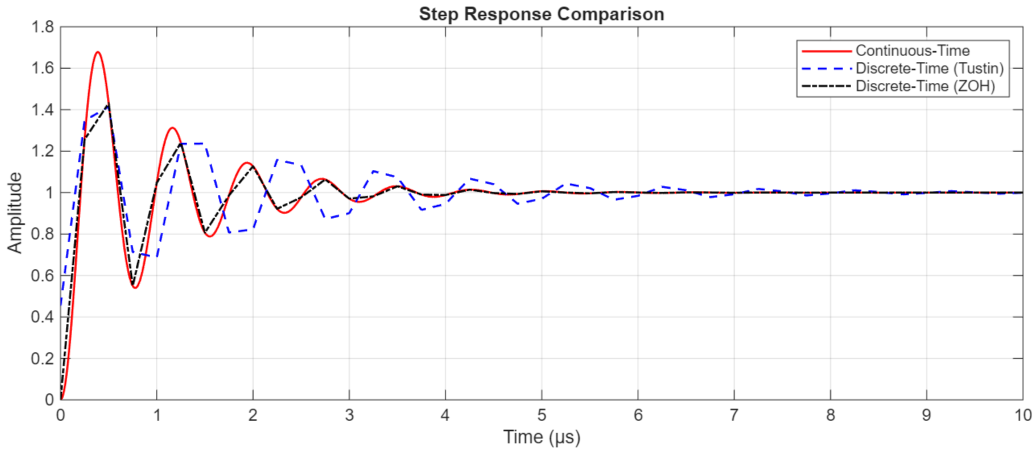

Figure 2.2: Step response comparison between continuous-time model and discrete-time models obtained with ZOH and Tustin methods

The tustin model oscillates for longer than the continuous-time model, which is not the case for the ZOH model. The ZOH model is therefore chosen for the following steps. The discrete-time state space matrices are:

$$A_d = \begin{bmatrix} -0.2560 & 28.7752 \\ -0.0173 & -0.4286 \end{bmatrix}, \qquad B_d = \begin{bmatrix} 1.2560 \\ 0.0173 \end{bmatrix}, \qquad C_d = \begin{bmatrix} 1 & 0 \end{bmatrix}, \qquad D_d = 0 \tag{2.9}$$

The poles of the discrete-time system are also computed by calculating the eigenvalues of the matrix A_d and the system is stable as they are in the unit circle:

$$\begin{cases} p_1 \approx -0.3423 + 0.6995i \\ p_2 \approx -0.3423 - 0.6995i \end{cases} \tag{2.10}$$

To determine whether the system is observable, we can compute the rank of the observability matrix $\mathcal{O}$:

$$\mathcal{O} = \begin{bmatrix} C_d \\ C_d A_d \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ -0.2560 & 28.7752 \end{bmatrix} \tag{2.11}$$

Which is of rank 2, meaning that the system is observable.